A Traveler's Guide to Spacewar!

For the curious visitor who wants to read one of computing's founding artifacts — the 1962 PDP-1 space-combat game — without a machine-room badge

---

Before You Arrive: What Is Spacewar!?

Spacewar! is a two-player space duel written for the DEC PDP-1 at MIT in 1961–62. It was conceived by Steve Russell, Martin Graetz, and Wayne Wiitanen, and implemented principally by Russell, with major contributions from Peter Samson (the star background), Dan Edwards (gravity and the outline compiler), and Graetz (hyperspace, in a slightly later version); Alan Kotok, Steve Piner, and Bob Saunders also contributed to its realization. Two ships — known to players as the *wedge* and the *needle* — orbit a central star that pulls on them with inverse-square gravity. Each player steers, thrusts, and fires torpedoes; the loser explodes in a shower of phosphor dots.

It is widely regarded as the first influential digital video game: it ran in real time, on a graphics display, under continuous player control, and its source circulated freely — copies of the paper tape traveled to many PDP-1 installations, and versions soon appeared on other research computers with programmable CRT displays.

A note on what you're holding. This particular file is `spacewar 2b, 2 apr 62` — and the header says so plainly: it was *reconstructed from disassembly by Norbert Landsteiner in 2014* and is **not** an authentic source listing. An authentic source for the nearly identical version of 25 March 1962 survives (linked in the header, at masswerk.at). The reconstruction is intended to correspond to the surviving binary/disassembly at the instruction level, and Landsteiner's `//` comments helpfully flag where this version differs from the later version 3.1 (e.g., the random-number constants). The original `/` comments — "crock explosion," "now go bang," "here put more bells and whisles" — are period text.

The whole game is one file: roughly 2,000 machine words of MIDAS macro-assembly for an 18-bit PDP-1 configuration with 4,096 words of memory and no automatic multiply/divide option.

The Map: Six Districts

Unlike a system of many source files, Spacewar! is a single scroll. But it reads as distinct neighborhoods, laid out roughly in this order:

```
[The Phrasebook]      MIDAS macro definitions — a tiny language for the game
[The Math Quarter]    imported sine/cosine, multiply, divide, square root
[The Shipyard]        the outline compiler — code that writes display code
[The Heavens]         the central star + Samson's "Expensive Planetarium"
[City Hall]           the main loop: object tables, collisions, frame timing
[The Combat District] spaceship control, gravity, torpedoes, explosions
```

Then two appendices: the ship outline data, and several pages of real star-catalog data.

District 1: The Phrasebook — the macros

The file opens by defining its own vocabulary. MIDAS macros (`define ... term`) wrap recurring instruction patterns into named idioms:

```
define count A,B
    isp A
    jmp B
    term

define swap
    rcl 9s
    rcl 9s
    term
```

`count` is a loop primitive: increment a (negative) counter and jump back until it crosses zero. `swap` exchanges the two CPU registers (AC and IO) by rotating the combined 36-bit pair 18 places. Later come domain-specific macros: `dispatch` (a computed jump table), `random` (a shift/XOR/add pseudorandom generator), `scale`, `diff` (a position/velocity integration step).

Before any of that, four definitions like this one:

```
szm=sza sma-szf    // skip on zero AC or on minus AC
```

> **★ DON'T MISS**
> That line is *arithmetic on opcodes*. PDP-1 "operate" and "skip" instructions are microcoded: each option is a bit in the instruction word, so you can compose new instructions by literally adding and subtracting existing mnemonics. `sza sma-szf` adds the skip-on-zero and skip-on-minus bits and removes a flag bit, yielding a new

> instruction: "skip if AC ⩽ 0." The instruction set itself was treated as a medium to hack on.

District 2: The Math Quarter — imported goods

This version targets a PDP-1 without the automatic multiply/divide option — the header says so explicitly — so the file carries its own arithmetic library, and the comments are honest about where it came from:

- `/sine-cosine subroutine. Adams associates` — a polynomial sine/cosine, 2.35 ms per call, credited to the consulting firm Adams Associates
- `/BBN multiply subroutine` and `/BBN Divide subroutine` — from Bolt Beranek and Newman, the other Cambridge institution with a PDP-1
- `/integer square root` — a bitwise shift-and-subtract root, needed for gravity

The multiply is a single unrolled line:

```
repeat 21, mus mp2
```

`mus` is "multiply step" — one bit of a shift-and-add multiply per instruction — and `repeat 21,` (21 octal = 17 decimal steps) stamps it out inline rather than looping, trading memory for speed.

> **★ DON'T MISS**
> This district is evidence that code sharing predates the game's own famous circulation. Spacewar! is assembled partly from routines that were already moving freely between MIT, BBN, and DEC's orbit. (Graetz's 1981 memoir "The Origin of Spacewar" tells the story of driving off to fetch the sine-cosine routines; the comment here records the upstream author.)

District 3: The Shipyard — the outline compiler

This is the most technically startling stop on the tour. The two ships are not stored as pictures or display lists. They are stored as *programs to be compiled*.

Each ship outline (`ot1`, `ot2`, near the end of the file) is a string of 3-bit direction codes packed six to a word:

```
ot1,    111131
    111111
    111111
    111163
    ...
    700000
```

At game start (`a3`), the code calls `jda oc` — the **outline compiler** — once per ship. `oc` walks the 3-bit codes and *emits PDP-1 instructions into memory* (via the `plinst` macro: "plant instruction"), producing a straight-line display subroutine custom-built for that hull shape. Codes 1–5 dispatch to small templates (`oc1`–`oc5`) that emit "step one plotting position in this direction and display a point" — where each direction is expressed in terms of the ship's current sine/cosine values, so the generated routine draws the outline *rotated to the ship's heading* using increments computed once per frame. Code 7 emits an epilogue that flips the sign of the step increments and replays the outline, drawing the mirrored other half of the (symmetric) ship — so the data only encodes half the hull.

The motive was speed: per Graetz's account, Dan Edwards built the compiler to claw back enough cycles per frame to afford real gravity. The compiled routines are pure straight-line code — no interpretation loop at display time.

> **★ DON'T MISS**
> Run the dates past yourself again. This is runtime code generation — generating specialized machine code from a compact data description to meet a performance budget — in a *game*, in early 1962. The modern analogy would be JIT-like specialization; here it has no name, it's just what was necessary to make two rotating ships and gravity fit in the frame.

District 4: The Heavens — the star and the Planetarium

Two pieces of sky share this district.

The heavy star (`blp`/`bpt`) sits at screen center. It is drawn each frame as a small randomized scatter of points whose pattern is re-rolled from the `random` generator — which is why, on a real Type 30 display, it appears to burn and shimmer rather than sit there as a fixed glyph.

The background is Peter Samson's *Expensive Planetarium* (his comments are initialed `prs`, dated 3/13/62 — the name riffs on MIT's "Expensive Typewriter" editor, itself a joke about using a $120,000 computer for mundane work). It is not decorative noise: the second half of the file is a genuine star catalog. Stars are entered with the `mark` macro and annotated with their Flamsteed designations and names:

```
1j, mark 1537, 371  /87 Taur, Aldebaran
    mark 1762, -189 /19 Orio, Rigel
    mark 1990, 168  /58 Orio, Betelgeuze
    mark 2280, -377 /9 CMaj, Sirius
```

The sky is modeled as a band 8192 units of right ascension wide (note the explicit `decimal` pseudo-op — this section abandons octal). The display routine (`dislis`/`bck`) shows whatever 1024-unit window currently overlaps the screen, and the window's left margin `fpr` creeps along and wraps, so the heavens slowly scroll past — a working planetarium of the real equatorial sky between roughly 22½°N and 22½°S, described in a contemporary account as accurate to about fifth magnitude, while Landsteiner characterizes the implemented tables as the stars of the first four magnitudes.

Brightness is done with *time*, not intensity codes: the main loop calls the first-magnitude table (`1m`) twice every frame, the next tables every frame, every other frame, and every fourth frame respectively. Brighter stars are simply refreshed more often, and the phosphor does the rest.

> **★ DON'T MISS**
> Sense switches 3 and 4 select among `MOVING STARS, NORMAL / FAST / STATIONARY / NO STARS` (see the header). The console's six toggle switches are the game's entire options menu — switch 1 picks the rotation model, switch 2 the gravity strength, switch 5 what happens if you fall into the star, switch 6 whether the star exists at all. Configuration without a single line of UI code.

District 5: City Hall — the main loop

`ml0`/`ml1` is the heart of the program, and it is organized around a structure modern game programmers will recognize instantly. There is one table region, `mtb`, holding up to `nob` objects (`nob=30` octal — 24 decimal: two ships plus torpedoes and explosions), and a set of *parallel arrays*, one per property, each `nob` words apart:

```
nx1=mtb nob     / x position
ny1=nx1 nob     / y position
na1=ny1 nob     / explosion/torpedo timer
nb1=na1 nob     / instruction count of calc routine
ndx=nb1 nob     / dx ... then dy, angular velocity, angle,
                /  acceleration, fuel, torpedoes remaining,
                /  outline address, old control word, spares
```

The first word of each object's entry is the address of its *calc routine* — `ss1`/`ss2` for the ships, `tcr` for a torpedo, `mex` for an explosion — and a zero means "slot free." An object changes what it *is* by having a different routine address stored into it: when two objects collide, the loop simply overwrites both of their calc-routine pointers with `mex`, and they are now explosions. Behavior-as-data, in 1962.

Collision detection is a brute-force pairwise sweep using cheap arithmetic only: collide if $|\Delta x| < \varepsilon$, $|\Delta y| < \varepsilon$, and $|\Delta x|+|\Delta y| < 1.5\varepsilon$ ($\varepsilon = 4096$ of 131072 screen units) — a square clipped to a diamond, no multiplication anywhere near it.

> **★ DON'T MISS**
> The frame timer. Each calc routine has a cost in instructions recorded in the `nb1` array (a ship is budgeted 2000 octal = 1024 instructions); the loop sums what was actually spent into `\mtc`, which started the frame at −4000 octal. The last thing the loop does is:
>
> ```
> count \mtc, .    / use up rest of time of main loop
> ```
>
> `jmp .` — jump to yourself — spinning in place until the budget is exhausted. The frame takes the same wall-clock time whether the sky holds two ships or two dozen torpedoes: a fixed frame rate, enforced by deliberately wasting the leftovers.
> And note what happens if a torpedo launch finds *no* free slot: `hlt / no space for new objects`. The program doesn't drop the torpedo — it halts the entire computer. Out-of-memory handling, 1962 style.

District 6: The Combat District — ships, torpedoes, explosions

The ship calc routine (`ss1`/`ss2` → `sr0`) is the longest stretch of game logic, and it runs once per ship per frame:

1. **Read the controls.** Via `jsp i \cwg`, indirecting through whichever input routine was installed at startup: `mg1` (`iot 11` — the custom control boxes Alan Kotok and Bob Saunders built) or `mg2` (the console test-word switches, entry point `a1`). Four bits per player: rotate left, rotate right, thrust, fire.
2. **Integrate rotation.** Sense switch 1 picks between angular *acceleration* with momentum ("inertial") and direct angular velocity — which the header calls **BERGENHOLM ROTATION**.
3. **Gravity.** Distance from the star via the square-root routine, an inverse-square acceleration via multiply and divide, scaled by sense switch 2 (light/heavy). Fall inside the minimum radius and you reach `pof`: depending on sense switch 5 you are either pinned dead at the center (`/ spaceship in star`) or flung to coordinates `377777` — the far corner, the "antipoint" (`/ now go bang`).

4. **Draw.** Position the beam, then `jmp` into the compiled outline routine from District 3. If the thrust bit is down, append the exhaust flame: a short randomized tail of points stepped backward along the heading (`sq7`).
5. **Torpedoes.** Firing requires an *edge*, not a held button — the new control word is ANDed with the complement of the old one (`mco`), so you must release and re-press. Each launch claims a free object slot, sets its calc routine to `tcr`, gives it the ship's velocity plus muzzle velocity along the heading, and starts two timers: 40 octal (32) frames before your tube "cools" enough to fire again, and a torpedo lifetime of 300 octal (192) frames. You carry 40 octal (32) torpedoes. Notably, `tcr` applies no gravity — torpedoes fly straight, which players learned to exploit.

Explosions are the routine the author labeled `/ crock explosion` — it scatters a random, slowly thinning cloud of points around the death site, for a duration derived from the colliding objects' instruction-count weights, then zeroes the slot.

> **★ DON'T MISS**
> Two lines, one of them the most famous comment in the file:
>
> ```
> / here put more bells and whisles, like hyperspace
> ```
>
> It sits exactly where the hyperspace ("panic button") feature would shortly be installed — a to-do comment, spelling intact, marking the program as a living draft. This version is dated April 2, 1962, weeks before the game's celebrated public showing at MIT's Parents' Weekend, April 28–29, 1962 (Graetz's memoir recalled a May Science Open House; Landsteiner's dating corrects this).
> And "BERGENHOLM ROTATION" in the header: the Bergenholm is the inertialess drive from E. E. "Doc" Smith's *Lensman* novels — the space-opera serials the authors explicitly cited as the game's inspiration. The science fiction is written directly into the configuration switches.

A Day in the Life: One Frame, End to End

Sixty-ish times a second, the machine does this:

1. **`ml0`** resets the frame budget (−4000 octal) and re-arms the table pointers.
2. **`ml1` sweep** walks all 24 object slots. For each live pair it runs the diamond collision test; any hit converts both objects' calc pointers to `mex` and sets the explosion timer.
3. **Calc & display** — each live object's routine runs: ships read controls, integrate rotation and gravity, update position, and execute their compiled outline routine (plus flame, plus torpedo logic); torpedoes integrate velocity and plot a point; explosions plot their dwindling debris cloud.

4. `background` refreshes the Expensive Planetarium — first-magnitude stars twice, dimmer tables on their alternating schedules — and nudges the sky window one unit westward every few dozen frames.
5. `jsp blp` redraws the flickering central star (unless switch 6 says there isn't one).
6. `count \mtc, .` burns whatever instruction budget remains, pinning the frame length.
7. `jmp ml0` — `/ repeat whole works`.

All positions are 18-bit one's-complement numbers that simply wrap, so the playfield is a torus: fly off the right edge, reappear on the left.

Practical Notes for the Independent Traveler

On the machine. The PDP-1: 18-bit words, 4K words of core, two registers (AC and IO), one's-complement arithmetic, ~5 μs per instruction. The display is the Type 30 CRT: the `dpy` instruction plots a single point at the coordinates held in AC (x) and IO (y); `ioh` waits for the completion pulse. There are no frames or buffers — the picture exists only because the loop redraws every point before the phosphor fades.

On MIDAS notation. Five things unlock most of the file:

- Numbers are **octal** by default (`nob=30` is 24; the star section explicitly switches with the `decimal` pseudo-op)
- `/` begins an original comment; `//` marks Landsteiner's 2014 annotations; `\name` is a macro-generated variable
- `(value` is an inline literal — the assembler stores the constant and the instruction references it
- `law N` loads a small literal into AC; `law i N` loads −N — which is why counters everywhere are negative numbers counted *up* with `isp`
- `dap X` **— deposit address part — is the master idiom.** It writes the AC's low bits into the address field of instruction `X`. Every subroutine return, every table walk, every indexed access is done by rewriting instructions in place. The base PDP-1 has no index registers and no stack; self-modifying code isn't a trick here, it's the calling convention.

On the patch space. `p, . 200/ space for patches` reserves 128 words of empty memory, and `. 5/` follows each ship outline. When your program is a physical paper tape, you fix bugs by patching jumps into reserved holes rather than reassembling — the file ships with its own repair margins.

On running it. This source assembles and runs today under PDP-1 emulation; the Critical Code Studies project (github.com/spacewar1962) bundles a browser-based emulator alongside the source and assembler listing, so you can watch the Type 30 output while reading the code that generates it. Landsteiner's masswerk.at hosts the disassembly this file was reconstructed from and the authentic 25 March 1962 listing.

What Makes Spacewar! Worth Visiting

The headline facts — first influential video game, freely circulated tape, ancestor of an industry — are all true, but they're about the game. The *code* rewards a visit for a different reason: nearly every structural decision is a negotiation with a hard budget of 4K words and one frame's worth of instructions, and the solutions are recognizably the ancestors of standard practice. Parallel property arrays indexed by object slot; behavior switched by swapping a routine pointer; a fixed frame rate enforced by burning the slack; collision via cheap norms; and, most strikingly, a compiler embedded in the program to turn shape data into specialized display code. None of this is presented as architecture — there is no abstraction layer, no module boundary, just 2,000 words doing exactly what the machine and the deadline demanded.

And the margins are full of people: Samson's initialed star catalog, the Lensman reference in a switch setting, the "crock explosion," the standing invitation to add "more bells and whisles." The program reads less like a finished product than like the lab notebook of a group having an extremely good time.

Provenance and Sources

Claims in this guide about code behavior come primarily from `spacewar_2b_2apr62.txt`, Norbert Landsteiner's 2014 reconstruction of Spacewar! 2B from disassembly. That file is not an authentic period source listing, and Landsteiner marks it as such; however, it is tied to the surviving 2 April 1962 binary/disassembly and is accompanied at masswerk.at by the authentic 25 March 1962 listing, the starfield data, and related Spacewar! sources. Historical attributions and development history follow J. M. Graetz, "The Origin of Spacewar" (*Creative Computing*, 1981), supplemented by Landsteiner's documentation. Some close readings — notably of the generated outline code and the explosion timing — involve interpretation of dense self-modifying PDP-1 assembly rather than claims stated explicitly in the comments. The broader framing accompanies the Spacewar! Critical Code Studies project of David M. Berry and Mark C. Marino.

References

- **Spacewar! source archive** (origin of this file and its companions — the reconstructed

2 Apr 1962 source, the authentic 25 Mar 1962 listing, the 2B disassembly, and Peter Samson's starfield data): Norbert Landsteiner, *Spacewar! Sources*, masswerk.at — https://www.masswerk.at/spacewar/sources/

- Norbert Landsteiner, *Inside Spacewar!* and related Spacewar! documentation, emulation, and dating research (including the Parents' Weekend correction and the quoted contemporary account in *The Tech*), masswerk.at — https://www.masswerk.at/spacewar/
- J. M. Graetz, "The Origin of Spacewar," *Creative Computing*, August 1981 — the principal first-person account of the game's conception, development, and credits
- David M. Berry and Mark C. Marino, *Spacewar! (1962): A Critical Code Studies Reading* — https://github.com/spacewar1962 and project site https://spacewar1962.github.io/spacewar/
- DEC, *PDP-1 Handbook* (Programmed Data Processor-1 manual) and the MIT *MIDAS* assembler documentation — background for the instruction set and assembler notation described in the Practical Notes